European Journal of Operational Research 61 (1992) 306–317
North-Holland

Theory and Methodology

A logic approach to the resolution of constraints in timetabling

Le Kang and George M. White
Computer Science Department, University of Ottawa, Ottawa, Ontario, K1N 6N5, Canada

Received May 1990; revised August 1990

Abstract: The construction of timetables involving three or more variables taking values from domains having thousands of values with several dozen constraints having different priorities is a problem common to many organizations. Various heuristic solutions have been proposed using results based on graph theory, mathematical programming and manual methods. We propose an approach based on logic programming using the first order predicate calculus. An algorithm has been developed, implemented in WPROLOG, and thoroughly tested on an Amdahl 5880 under VM HPL.

Keywords: Timetabling; scheduling; logic programming; constraint programming; expert systems; truth maintenance systems

1. Introduction

Timetabling or the casting of timetables is a common activity in organizations where groups of people must schedule a number of meetings during some interval of time using a set of resources such as meeting rooms. Usually there is also a set of constraints which stipulate such things as the amount of time allowed per meeting, the required attendance at the meetings and the mix of resources. Scheduling the annual meeting of a learned society, the use of dental chairs in a multi-practice clinic and the timetabling of university classes are examples of this activity.

The problem would be relatively easy were it not for the presence of the constraint set. Critical conflicts, or first order constraints, are conflicts which are generally agreed to be unacceptable: the scheduling of two meetings to take place in the same room at the same time, or the requiring of participants to be in two places simultaneously. Non-critical conflicts, or second order constraints, are those which one would like to avoid if at all possible but which one is prepared to live with if there are no other choices. These conflicts, unlike the first type, are very of local origin and may vary greatly depending on the exact circumstances of time and place.

First order constraints must be avoided and systems which construct timetables must be programmed to eliminate any possibility of their occurrence. Timetables which satisfy all their first order constraints are called *feasible* timetables. Second order constraints may be permitted *if absolutely necessary*, but even then, should be kept to an absolute minimum. Often, some of the second order constraints are more odious than others. The least desirable of this latter category must be avoided with more rigour than the less objectionable ones. The objective function approach is generally unsuitable for problems of this complexity due to the difficulty or impossibil-

0377-2217/92/$05.00 © 1992 – Elsevier Science Publishers B.V. All rights reserved

ity of quantifying some of the factors. e.g. What are the relative penalties which should be imposed on using a room which is larger than necessary versus having to walk to a better sized room some distance away. In addition there are difficulties of a political nature which are created when attempts are made to quantify factors of this type.

The work reported here is a study of constraint resolution with both first and second order constraints examined in the context of casting university course timetables. The motivation for this study arose from the requirement for institutions to cope with ever increasing use of resources using frozen budgets which barely cover inflation. The manually generated timetables which are in use almost everywhere at the present time are not optimized and are often not even feasible. Thus a more efficient use of existing resources was the chief goal. We would also like to reduce the large number of man-hours required to prepare and modify the timetable.

Unfortunately, the most obvious methods of gaining greater efficiency such as packing more and more students into smaller and smaller spaces in undesirable for other reasons. So while wanting greater efficiency in terms of room utilization and setting usage per student, the system must also handle the demands for an increased choice of courses for all student programs, unavailability of professors during certain periods due to meetings and outside involvements, geographical separation of buildings throughout the campus and a host of other factors.

There is as yet no generally acceptable method for solving problems of this complexity. An easier related problem, the examination scheduling problem, has been much more intensively studied and several approaches exist. These approaches have been extended, with varying degrees of success to the timetabling problem. The first of these, the graph colouring problem, is the oldest and has achieved a certain measure of success. This is followed by various mathematical programming methods such as column generation which are usually used for problems which require some degree of optimization. Semi-interactive methods which use human timetabling experts to augment knowledge embedded in the computer system have found some use.

The approach studied here is based on logic programming and consists of the resolution of constraints framed in first order logic clauses. This is a technique which has only just emerged and at the time the project was begun there was virtually nothing known about how successful this approach would be in the solution of problems of this size and complexity. There were three reasons why the approach seemed to be worthwhile:

Modularity. The logic approach permits changes to the program to be made by adding or removing clauses. A clause describing one constraint is independent of the others and therefore changes can be made easily without modification to other parts of the program.

Portability. Programming was done in a relatively simple variant of a common logic programming language, W-PROLOG.

Feasibility. Large numbers of complicated constraints are more easily formulated within the first order logic than they are using other, more common, techniques because they do not require any procedural specifications [7]. Thus the complex requirements of real timetables can be faithfully captured by the computer system without practical compromises, yielding results which are truly feasible.

We present below a definition of the timetabling problem via two sub-problems, time scheduling and room scheduling. A discussion of four methods used to cast timetables then follows and one of these, the first order logic method, is discussed in detail. The formulation of the timetabling problem in logic is presented along with experimental results and implementation details. The testing procedures are described and finally, conclusions are drawn.

2. The timetabling problem

The timetabling problem has been studied by researchers since the early 60s and a good deal of relevant literature [17] has appeared. The approaches have varied from computer based cloning of manual book-keeping systems to those based on the first order predicate calculus. In an effort to reduce the complexity of the problem, timetabling is often carried out in two separate stages; *time scheduling* where course-teacher pairs are scheduled into a fixed number of time periods and *room scheduling* which assigns class-

rooms to classes. The well known examination timetabling problem [21] is an example of the first stage.

2.1. Time scheduling

A formulation of the time scheduling problem can be given in the form proposed by De Werra [5].

Let $C=\{c_1,\ldots,c_m\}$ be a set of classes and $T=\{t_1,\ldots,t_n\}$ a set of teachers. There is an $m * n$ requirements matrix $R=(r_{ij})$ where r_{ij} is the number of lectures involving class c_i and teacher t_j. Let $x_{ijk}=1$ if class c_i and teacher t_j meet at period k for a lecture which is not a preassignment; $x_{ijk}=0$ otherwise. The timetable constraints are:

$$\sum_{k=1}^{p} x_{ikj}=\bar{r}_{ij} \quad (i=1,\ldots,m;\ j=1,\ldots,n),$$

$$\sum_{j=1}^{n} x_{ikj}\le b_{ik} \quad (i=1,\ldots,m;\ k=1,\ldots,p),$$

$$\sum_{i=1}^{m} x_{ikj}\le c_{jk} \quad (j=1,\ldots,m;\ k=1,\ldots,p),$$

where

$$\bar{r}_{ij}=r_{ij}-\sum_{k=1}^{p} r_{ijk},$$

$b_{ik}=1$ if c_i is available and not preassigned to period k, and 0 otherwise,
$c_{jk}=1$ if t_j is available and not preassigned to period k, and 0 otherwise,
$r_{ijk}=1$ if there is a preassignment meeting of class c_i and teacher t_j during period k, and 0 otherwise. Although in some special cases it can be decided if such a timetable exists or not, [5] Even, Itai and Shamir have proved that the problem of deciding whether a solution exists for the scheduling problem is NP-complete [8].

2.2. Room scheduling

The assignment of classrooms to classes is often studied separately from the scheduling of teachers and classes. The formulation of the problem is normally expressed in terms of an occupancy variable, x_{ij}, that is equal to 1 when class c_i is assigned to room r_j. The constraints for this problem are [3]

$$\sum_{j=1}^{m} x_{ij}=1 \quad (i=1,\ldots,n),$$

$$\sum x_{ij}\le 1 \quad (j=1,\ldots,m;\ k=1,\ldots,p)$$

where p_k represents the set of all classes that meet in period k. The first constraint ensures that every class is assigned to some room while the second constraint prevents any room from having more than one class in the same period.

2.3. Timetabling

The complete timetabling problem involves a set of class-teacher pairs, a set of time periods, i.e. intervals during which classes are given, a set of student programs representing the sets of classes that certain groups of students have to take, and a set of classrooms. The elements of these sets should be scheduled in such a way that:

(a) no teacher or class is required to be in more than one place during the same time period

(b) all classes are scheduled

(c) all classrooms are large enough to hold the classes assigned to them.

These constraints may be expressed formally as follows:

Let there be m time periods, $k=1,\ldots,m$; n_2 teachers, $j=1,\ldots,n_2$; n_1 classes, $i=1,\ldots,n_1$; and n_r rooms, $l=1,\ldots,n_r$. There is a requirements matrix $R=(r_{ij})$ and two vectors, E is an n_1-vector whose i-th element is the enrollment in class i and C is an n_r-vector whose i-th element gives the capacity of room i. Determine whether there is a function

$$S=\{x_{ijkl}\}:\{1\ldots m\}\times\{1\ldots n_1\}\times\{1\ldots n_2\}\times\{1\ldots n_r\}\to\{0,1\}$$

where $x_{ijkl}=1$ iff teacher j meets class i in room l during period k such that:

$$\sum_l\sum_k x_{ijkl}=r_{ij}, \tag{1}$$

$$\sum_j x_{ijkl}\le 1, \tag{2}$$

$$\sum_i x_{ijkl}\le 1, \tag{3}$$

$$\sum_l x_{ijkl} \leq 1, \tag{4}$$

$$\sum_j \sum_k e_i x_{ijkl} \leq \sum_j \sum_k C_l x_{ijkl}. \tag{5}$$

Constraint (1) requires that the number of meetings of class i and teacher j conforms to the specifications of the requirements matrix; constraint (2) specifies that each period-class-room has at most one teacher involved; constraint (3) specifies that each teacher-period-room has at most one class in ti; constraint (4) specifies that each teacher-period-class occupies at most one room; and constraint (5) specifies that all rooms are large enough to contain the classes scheduled into them.

It is evident that the timetabling problem is inherently complex. In real-life situations this complexity becomes very difficult to manage due to a number of complicating factors.

(i) When the number of courses becomes larger than about 1000, the number of constraints becomes so large that many algorithms become impractical or impossible to implement [12]. Furthermore if secondary constraints such as teacher preference and availability are taken into account the time may become unacceptably long and even then, no solution is guaranteed.

(ii) Building a timetable involves some psychological as well as pedagogical constraints. A good timetable should incorporate not only a feasible arrangement of physical resources, such as classroom and other academic utilities but should also attempt to resolve psychological quirks such as a teacher wanting to teach only in a certain room at a certain time. Furthermore, almost any resolution of timetabling problems requires some compromises, between departments, for example [13].

(iii) Local conditions vary so much among institutions that a single program is not likely to be useful to more than one of them without modification. Not only are the requirements different, but even the format of the time periods varies from one institution to the next.

The second order constraints are those which may be relaxed if, and only if, this is required to obtain a feasible solution. There should be as little relaxation as possible. In the present work each secondary constraint has been given a precedence level which denotes its priority of being kept in case relaxation is necessary. The higher the priority, the less likely it will be relaxed. The constraints are relaxed in groups according to the group's priority level.

Priority 1. Courses should be scheduled into classrooms in the building as close as possible to the host department or the building specified for them.

Priority 2. Courses should be scheduled into rooms of the seating type (fixed chairs, movable chairs, etc.) specified for them. Courses should be scheduled into rooms of the class type (amphitheatre, seminar room, etc.) specified for them.

Priority 3. Courses should be scheduled into rooms of the furniture type (movable tables, fixed tables, etc.) specified for them.

Priority 4. Courses should be scheduled into classrooms which are just large enough to completely contain all the students enrolled in those courses. Recommended courses of a student program should be scheduled in such a way that there are the fewest time conflicts among them.

Priority 5. Courses should be scheduled into the time zone (morning, noon, afternoon) specified for them. Night courses are always scheduled at night. Friday afternoon shall be used only after all other scheduling possibilities are exhausted.

3. Timetabling algorithms

The algorithms used to construct timetables can be classified into four types.

1) those based on graph theory
2) those based on mathematical programming
3) those using an interactive approach based on human expertise
4) those based on first order logic

3.1. Graph colouring algorithms

The algorithms in this class are based on or related to graph colouring problems. For the simple course scheduling problem, it can be posed as follows:

1) each course is represented by a vertex;
2) an edge connects two vertices if the two corresponding courses will be taken by at least one student or given by the same teacher.

The algorithm then finds a method of 'colouring' the nodes of the graph such that no two

nodes joined by an edge have the same colour and the minimum number of colours are used.

Some of the algorithms, e.g. [2,15] will find a colouring of the graph with p or fewer colours if it exists. These are exact algorithms and are feasible only if the number of courses is small because the problems are NP-complete and the algorithms enumerate implicitly all possible colourings.

Various heuristic algorithms have been developed for graph colouring in which a colouring is found but the number of colours used is not guaranteed to be a minimum. One of these is invariably used in the real-world construction of timetables when this type is implemented because reasonable solutions can be obtained in a reasonable amount of time.

3.2. Mathematical programming algorithms

If the timetable problem is considered to be one of reducing existing constraint violations, it can be presented as an optimization problem. The major drawback of this approach is that the number of variables and constraints becomes unmanageably large for practical problems. In an attempt to reduce the problem size, researchers have proposed several different models, heuristic methods, [1,18] network models, [14] Lagrangian relaxation, [3,19] and quadratic assignment [10].

3.3. Interactive algorithms

A third class of algorithms uses a human intermediary to suggest parts of the solution while the computer verifies that no constraints have been violated. The amount of computer 'timetable knowledge' in various systems varies from zero to the complete set of first and second order constraints [20]. An interactive system is usually designed to support two phases of construction: a preparatory phase which is batch oriented and creates a database and an interactive phase which includes one or more interactive sessions during which the database created earlier is modified by the users. When all modifications are complete, the timetable is published.

To accommodate last minute changes to the timetable, real-world systems often have an interactive algorithm as the last portion of a system based on either of the graph colouring or mathematical programming approaches. This eliminates a complete rerun of the algorithm with the updated data which would produce a wholly new timetable completely different from the previous result.

The fourth type of algorithm, that based on the first order logic, is discussed in the following section.

4. The logic model

The logic model used here consists of a set of axioms in Horn Clause form, the axioms taking the role of procedures to be executed by an interpreter. Computation is done by top-down inference using a proof procedure. Logic programming systems have a built-in, general pattern matching facility based on Robinson's unification principle [16].

The Horn Clause subset of predicate logic is used to express the constraints. A Horn Clause is either a simple assertion (e.g. George is the father of Christine) or an implication (e.g. x inherits y if x is a child of z and z has y). Every *assertion* is a simple fact which expresses a relation among entities, whereas every *implication* expresses the result of a condition and has the form

$$A \text{ if } B_1 \text{ and } B_2 \text{ and } \cdots \text{ and } B_n$$

where conclusion A and conditions B_i are atoms.

Pattern matching and backtracking are two important facilities of logic systems used to find solutions for queries (proving goals). A goal or query is proved either by matching it with a fact, (an assertion), or by matching it with the conclusion part of an implication and then proving all the conditions of this implication. This matching is called *pattern matching*.

Logic systems must be able to try a number of combinations if necessary to prove the goal, or to find ways it can be proved. The most popular method used by logic systems in implementing this facility is called *backtracking* or *unification*. Two patterns, a goal and an assertion, are matched or unified only if they have the same name and number of arguments and each pair of corresponding arguments is unified. Arguments are unified according to the following rules:

(i) If one of the arguments is a variable with an unspecified value and the other is a variable

with a value, they are unified by assigning the value to the variable.

(ii) If both the arguments are variables without values, their equality is recorded without assigning any value to them. Later, when any two of these variables is assigned a value, the other will be assigned the same value.

(iii) Otherwise, the arguments can be unified only if they are both constants, or both structures.

a) If both arguments are constants, they are unified if they match exactly.

b) Two structures are unified if they have the same name and number of arguments, and each pair of corresponding arguments is unified.

Backtracking enables logic systems to explore all the possible combinations in which a goal can be proved. The order in which the various parts of a query are specified does not affect the final result although it may affect the efficiency of the program.

The use of logic programming in computerized timetabling is a relatively new approach which has received little attention to date. A related research area is that sometimes called the constraint satisfaction problem, studied in the field of artificial intelligence [4,11]. A recent paper by Feldman and Golumbic [9] reported on a project which used constraint satisfiability algorithms for interactive student scheduling, a problem related to timetabling. The project was to determine which algorithms perform best in solving the student scheduling problem and under what conditions.

The student scheduling problem is the problem of searching for suitable schedules for students from the general university timetable. There are two types of constraints which must be satisfied and which can be relaxed according to certain priorities. After testing three algorithms, regular backtracking, forward checking, i.e. sorting of the variables by increasing order of their domain sizes after each stage, and word-wise forward checking, the authors concluded that with a larger number of courses the word-wise algorithm was clearly better than the other two and regular backtracking was the worst. If the number of courses was fewer than 19, the regular backtracking became the best. Also the order in which the variables were assigned values significantly affected the performance of the algorithms.

The smaller problem of assigning teachers to courses has been formulated both as an integer programming, IP, problem and as constraint satisfaction problem, CSP, by Dhar and Ranganathan [6]. Depending on the values of the data presented to the algorithm, solutions were found for both formulations although the IP solution time was far more volatile than the CSP time. In addition, the control of the reasoning process was greater for the CSP. Partial solutions were also easier to formulate in CSP than they were in IP.

A similar problem, the cutting-stock problem has been studied [7] both as a CSP and by using integer linear programming. The study's conclusion was that the logic approach was far easier to express and to program and was more efficient in its use of computing resources than was the linear programming approach.

The constraint satisfaction problem can be formally defined as:

Consider a finite set of variables, $V = \{X_1, \ldots, X_n\}$, which take values from their corresponding domains, $D = \{D_1, \ldots, D_n\}$. A constraint, $c(X_{i_1}, \ldots, X_{i_k})$ among k variables from V is a subset of the cartesian product, $D_{i_1} \times \cdots \times D_{i_k}$ which specifies which values of the variables are compatible which each other. Now, find a solution to the problem by assigning values to all the variables from their domains such that all the constraints are satisfied.

There are 5 variables taken into account in the timetabling system. These are:

Course. This variable represents the course to be timetabled whose domain contains all the courses, labs and seminars to be scheduled. A course has several properties: student enrollment, type, format, and professor.

Time. or time period, represents the time occupied by the course. Its domain, T, is the set of all the time blocks permitted in the time slot system. Most of the courses fall into one major time slot system although many other formats are tolerated. For this work we have adopted one standard time slot system in which course lengths are limited to multiples of 11/2 hours. One week is divided into 6 working days including Saturday and each day is divided into 9 one and a half hours blocks, except Saturday which has only one block. The domain of time, therefore, has 46 values.

Room. represents the classroom. Its domain, R, contains all the classrooms available to the system. The properties of room are room_type, furniture_type, seating_type, room_size and a list of reserved time blocks. For every classroom, R_i, there is a set of course-time pairs of the form $R_i(c_j, t_k)$ where c_j is the course taught in R_i at time t_k.

Professor. represents teachers for courses. Its domain, P, is the set of all professors teaching *course*. A *professor* may have a list of *time* values during which he cannot teach. For each professor, p_i, there is a set of courses, Q_i, which he teaches.

Student-program. represents student academic programs. This variable has two properties, required courses and recommended courses and will be written in the form $sp_i(s_1, s_2)$ where s_1 and s_2 represent respectively the list of required courses and the list of recommended courses. The domain of this variable, S, is the set of all student programs in all the departments and faculties at the University. Generally a group of students of the same year and studying the same major area of one department will be considered as being in the same academic program.

The logic model has 12 first order constraints and 6 second order constraints which are listed in the appendix.

5. Implementation details

A program incorporating the constraint resolving algorithm described here was implemented at the University of Ottawa on the university's Amdahl 5880 mainframe running under the VM HPL operating system. This platform was chosen because all the university's student data is stored there and the administrative officers have accounts on the system and can easily update input information and access the system.

The main program was written in Waterloo PROLOG (WPROLOG) with several utility programs written in C, for implementation efficiency. WPROLOG was chosen because it was the only Prolog engine available on the hardware platform and because its implementation was robust. WPROLOG is a simple but complete version of Prolog. The lack of non-standard features was considered to be a positive feature because it meant that programs developed in that language would be relatively portable across various interpreters and therefore could be used with little translation effort on a variety of execution platforms. The one weak point uncovered was the difficulty of interfacing with programs written in other languages (C in particular).

The program presently assumes that the timetable will schedule courses into standard units of time called a *timeslot* which has a fixed length of 11/2 hours. There are four course formats considered standard, $1 \times 1\frac{1}{2}$h, 1×3h, $2 \times 1\frac{1}{2}$h and 1×2h. Courses in the formal of 1×2h are permitted only if they start at 8:00 am, $\frac{1}{2}$h earlier than the normal starting time so as not to disrupt the block sequence.

In order to avoid the possibility of exhaustive searching when a course cannot be scheduled, an *equivalent reversing* procedure was developed. This is an alternative to complete backtracking as found in classical Prolog interpreters. When a course A becomes unschedulable, the procedure first chooses an equivalent course B which has already been successfully processed, i.e. has been given the proper assignments, based on some criteria. It then deletes B's assignments and tries to reschedule B after successfully scheduling A. If A cannot be scheduled after deleting B's assignment, the procedure will resume B's assignment and go on to choose another course according to the criteria and repeat the deleting-schedule-reschedule process.

For the purpose of equivalent reversing, courses satisfying the four criteria below are considered equivalent:

- their enrollments fall in the same size range,
- they are in the same timezone,
- they require the same room type,
- they request the same building.

In order to avoid infinite looping and to improve efficiency, nested equivalent looping was not used. The number of equivalent courses that can be selected in one reversing process has been limited to 4. Furthermore, assignments of completely pre-scheduled courses cannot be deleted. If the program cannot find an assignment for one course after the equivalent reversing procedure, it will put the course into a list of unschedulable courses which will be manually scheduled later.

The concept of a *size limiter* was used to incorporate the room size constraint. The avail-

Table 1
Size range chart

Size	Range number
> 301	19
201–300	17
151–200	15
101–150	13
71–100	11
51– 70	9
31– 50	7
19– 30	5
11– 18	3
1– 10	1

able stock of classrooms was partitioned into groups according to their size, each group having a *range number* (see Table 1).

When scheduling a course, the program tries to find classrooms that have the same range number as the course does. When all classrooms in this range have been tried and no suitable assignments found, the program may go up one or more ranges to find a room of greater capacity. The *size limiter* is the mechanism for achieving this. Each size range is associated with a size limiter. The size limiter of a size range indicates the number of larger size ranges which may be searched for a suitable room. A course of range, R, whose size limiter is L can only be scheduled into classrooms no larger than range $R + 2 * L$.

The size limiters are based on the results of previous runs and from previous experience.

6. Testing

Testing was begun using a small sample of data obtained from the Computer Science Dept. 140 course-teacher pairs were scheduled into 17 classrooms. The courses had enrollments ranging from 3 to 206 students. 26 of the pairs actually had teachers assigned. The others were lab periods, tutorial sections or courses for which the teacher was then unspecified. The 17 classrooms had capacities ranging from 21 to 210.

There were 40 constraints corresponding to the various degree programs offered by the department. A timetable was successfully generated with all classes scheduled. This example was small enough to be checked by hand and it was found that:

a) all constraints were satisfied,

Table 2
Size limiters

	Range number									
	1	3	5	7	9	11	13	15	17	19
Size limiter 1	0	0	1	0	0	0	0	0	0	0
Size limiter 2	1	1	2	1	2	1	1	1	1	0

b) the number of student-hours scheduled was equal to 4819,

c) the average seating usage was 76.7%. This figure is calculated by summing the total number of seats available in all the classrooms during those periods in which the room is used and then dividing this into the number of students occupying the room during those same periods.

A complete test of this program was accomplished using previous years' data from the entire university. The data set included 2268 course entries of which 96 were considered as pre-scheduled because they did not follow any of the four basic formats. Another 59 entries were considered pre-scheduled for other reasons.

There were 167 classrooms whose sizes varied from 4 to 474 and there were 363 student programs resulting in more than 14329 pairs of potentially time-conflicting courses. In addition, there were 256 professor timeblocks. For the two runs reported here, the only change made was that of the size limiter. In Table 2 the two used size limiters are shown. The most important results of the test runs are summarized in Table 3. An examination of these numerical factors shows the importance of the size limiter. Using size limiter 1 which is rather strict in its tolerance of excessive room size, a timetable is produced with 132 courses unscheduled. When the size limiter is set to more tolerant values, all the courses are scheduled and the execution time is somewhat

Table 3
Test results

	Test	
	1	2
Execution time (sec)	1427	1402
Pre-assigned entries	155	155
Number scheduled	1981	2113
Number unscheduled	132	0
Average seating usage (%)	85.5	80.2
Average room utilization – day classes (%)	55.4	58.5
Average room utilization – evening classes (%)	59.0	60.4

smaller. In the latter case, 167671 student-hours were all successfully scheduled.

The values of average seating usage and average room utilization are about 80% and 60%, respectively. With the current manual system the corresponding values are about 70% and 50%. Thus the efficiency of resource use has been improved.

However, these figures do not take into account the non-numerical constraints which are now being satisfied for the first time. The recommended course non-conflict requirement yields a greater selection of optional courses for students to take. In addition, the geographic requirement as defined by the host department means that there is less bodily movement across campus. Teachers can specify the periods when they do not wish to meet classes and can choose the type of classroom they wish to use.

It has proved impossible to compare the results of this algorithm with similar results derived from other computer methods because we have been unable to find other comparable computer systems which cast the complete timetable. Where we have comparable numeric results between this system and present manual methods, the computer results are unfailingly better.

7. Conclusions

Extensive testing of the algorithm has shown that it has achieved all its goals. All of the first order constraints and varying numbers of the second order constraints have been satisfied in solving a large constraint satisfaction problem with real application to large institutions. Either completely feasible solutions are found or certain courses are found to be unschedulable within the constraint set. In the latter case, the unschedulable courses are removed and a timetable is found for the remaining courses. The unschedulable courses are then scheduled manually. For real historical data, a feasible solution for all courses has been found. This is accomplished without consuming inordinate amounts of computer power, about 24 minutes of execution time on an Amdahl 5880.

Experience gained in constructing the system verified the importance of being able to change the number of type of constraints easily. It was found that an extended form of Horn Clause logic permitted complex constraints to be expressed moderately easily and to be readily translatable into WPROLOG.

Finally, the concept of equivalent reversing has proved itself useful in reducing the execution time of the program.

8. Acknowledgements

We are grateful for the unflagging interest and support of the Registrar of the University of Ottawa, Mr. George H. von Schoenberg, without whom this work could not have been undertaken. We were greatly assisted as well by Ms. Pauline Bélanger.

Appendix

The 12 first order constraints are listed below directly in terms of the Prolog predicates used to express them.

1) *No required courses may be scheduled at the same time.*

For all i, j, k,

no_common_member(T_j, T_k) ←
  is_a_member($sp_i(s_1, s_2)$, s) *and*
  is_a_member(c_j, s_1) *and*
  is_a_member(c_k, s_1) *and*
  $c_j \neq c_k$ *and*
  teaching_time(c_j, T_j) *and*
  teaching_time(c_k, T_k)

where $i = 1, \ldots, m$, $j = 1, \ldots, n$, $k = 1, \ldots, n$.

2) *No recommended course may be scheduled at the same time as a required course.*

for all i, j, k,

no_common_member(T_j, T_k) ←
  is_a_member($sp_i(s_1, s_2)$, S) *and*
  is_a_member(c_j, s_1) *and*
  is_a_member(c_k, s_2) *and*
  teaching_time(c_j, T_j) *and*
  teaching_time(c_k, T_k)

where $i = 1, \ldots, m$, $j = 1, \ldots, n$, $k = 1, \ldots, n$, $j \neq k$.

3) *No professor may teach more than one course at a time*

For all i, j, k,

no_common_member(T_j, T_k) ←

is_a_member(p_i, P) *and*
teacher_courses(p_i, Q_i) *and*
is_a_member(c_j, Q_i) *and*
is_a_member(c_k, Q_i) *and*
$c_j \neq c_k$ *and*
teaching_time(c_j, T_j) *and*
teaching_time(c_k, T_k)

where $i = 1,\ldots,q$, $j = 1,\ldots,n$, $k = 1,\ldots,n$.

4) *Day courses must be scheduled during the day and evening courses in the evening.*

For all j, k,

is_a_member(t_k, T') ←
is_a_member(c_j, C') *and*
teaching_time(c_j, T_j) *and*
is_a_member(t_k, T_j)

for all j, k,

is_a_member(t_k, b'') ←
is_a_member(c_j, C'') *and*
teaching_time(c_j, T_j) *and*
is_a_member(t_k, T_j)

where C' and C'' and subsets of C and contain, respectively, all of the n' day courses and all of the n'' evening courses; T' and T'' are subsets of T and contain all of the p' day slots and all of the p'' evening time slots respectively and

$j = 1,\ldots,n'$, $k = 1,\ldots,p'$,

or

$j = 1,\ldots,n''$, $k = 1,\ldots,p''$.

5) *All classrooms must be able to hold the students scheduled into them.*

For all i, j,

Rsize ≥ *Size* ←
is_a_member(c_j, C) *and*
enrollment(c_j, *Size*) *and*
teaching_room(c_j, R_j) *and*
is_a_member(r_i, R_j) *and*
room_size(r_i, *Rsize*)

where $i = 1,\ldots,m$, $j = 1,\ldots,n$.

6) *A classroom cannot be used if it has been reserved.*

For all i, j,

not(is_a_member(t, T_i)) ←
is_a_member(c_j, C) *and*
teaching_time_room(c_j, TR_j) *and*
is_a_member($tr_i(t, r)$, TR_j) *and*
reserved_time(r, T_i)

where $i = 1,\ldots,m$, $j = 1,\ldots,n$.

teaching_time_room(C, TR) is a predicate where C is a course and TR contains all the teaching time_room pairs of this course. Each member of TR has the form $tr_i(T, R)$ where i is the element index, T is one teaching time and R is the classroom.

7) *A professor cannot teach if he is unavailable.*

For all i, j,

no_common_member(T_j, T_i) ←
is_a_member(p_i, P) *and*
unavailable_time(p_i, T_i) *and*
teacher_courses(p_i, Q_i) *and*
is_a_member(c_j, Q_i) *and*
teaching_time(c_j, T_j)

where $i = 1,\ldots,q$, $j = 1,\ldots,n$.

unavailable_time(P, T) is a predicate where T is the set of all timeslots when professor P cannot teach.

8) *Certain courses can be assigned to a particular time and room.*

For all k, j,

is_a_member($tr_k(t, r)$, TR_j) ←
is_a_member(c_j, Z) *and*
reserved_value(c_j, Z_j) *and*
teaching_time_room(c_j, TR_j) *and*
is_a_member($tr_k(t, r)$, Z_j)

where Z_j is the set of n_j preassigned values of the time-room pairs of course c_j and $k = 1,\ldots,n_j$, $j = 1,\ldots,n$.

9) *Certain courses may be scheduled at the same time and place.*

For all k,

same_set(TR_j, TR_i) ←
is_a_member($pair_k(c_1, c_2)$, IP) *and*
teaching_time_room(c_1, TR_j) *and*
teaching_time_room(c_2, TR_i)

where IP is the set of all n_{IP} pairs of courses that should be given the same timeslots and classrooms and $k = 1,\ldots,n_{IP}$, $j = 1,\ldots,m$, $i = 1,\ldots,m$.

10) *Certain courses may be scheduled wholly or partially at the same time.*

For all k, some i,

is_a_member(t_i, T_a) ←
is_a_member($pair_k(c_1, c_2)$, PT) *and*
teaching_time(c_1, T_j) *and*
is_a_member(t_i, T_j) *and*
teaching_time(c_2, t_a)

where PT contains all n_{PT} pairs of courses that should overlap with each other in time and $k = 1,\ldots,n_{PT}$, $j = 1,\ldots,m$, $a = 1,\ldots,m$, $i = 1,\ldots,46$.

11) *Certain courses must be scheduled at the*

same time but in different rooms.

For all k, k_1, k_2,

$r_1 \neq r_2$ ←
is_a_member($pair_k(c_1, c_2)$, PTR) *and*
teaching_time_room(c_1, TR_{j_1}) *and*
teaching_time_room(c_2, TR_{j_2}) *and*
is_a_member($tr_{k_1}(t, r_1)$, TR_{j_1}) *and*
is_a_member($tr_{k_2}(t, r_2)$, TR_{j_2})

where PTR is the set of n_{PTR} pairs of courses whose teaching time should be the same but whose classrooms should be different and $k = 1, \ldots, n_{PTR}$, $j_1 j_2 = 1, \ldots, m$, $k_1, k_2 = 1, \ldots, n_j$.

12) *Courses scheduled twice per week must have at least one day between the two timeslots.*

For all j,

$t_1 - t_2 > 1$ *or* $t_2 - t_1 > 1$ ←
is_a_member(c_j, C) *and*
teaching_time(c_j, T_j) *and*
is_a_member(t_1, T_j) *and*
is_a_member(t_2, T_j) *and*
$t_1 \neq t_2$

where $j = 1, \ldots, m$, $t_1 \in T$, $t_2 \in T$.

In addition there are 6 second order constraints arranged in four groups. The higher the priority of the group, the less likely it is to be relaxed.

1) *Certain courses should be scheduled in the timezone specified.*

For all i, j,

is_a_member(t_i, $Tzone$) ←
is_a_member(c_j, C) *and*
course_timezone(c_j, $zone$) *and*
zone_time($zone$, $Tzone$) *and*
teaching_time(c_j, T_j) *and*
is_a_member(t_i, T_j)

where $i = 1, \ldots, 46$, $j = 1, \ldots, n$.

2) *Certain classrooms should be of the type required by the courses scheduled into them.*

For all i, j

$type' = type''$ ←
is_a_member(c_j, C) *and*
teaching_room_type(c_j, $type'$) *and*
teaching_room(c_j, R) *and*
is_a_member(r_i, R) *and*
room_type(r_i, $type''$)

where $i = 1, \ldots, l$, $j = 1, \ldots, n$.

3) *The rooms should be just large enough to contain all the classes scheduled into them*

for all i, j,

$per \geq Rate$ ←
is_a_member(c_j, C) *and*
enrollment(c_j, $Size$) *and*
teaching_room(c_j, R) *and*
is_a_member(r_i, R) *and*
room_size(r_i, $Rsize$) *and*
percentage($Size$, $Rsize$, per)

where $Rate$ represents the least seating usage rate admissible and $i = 1, \ldots, l$, $j = 1, \ldots, n$.

4) *Recommended courses of a student academic program should not conflict.*

For all i, j, k,

(no_common_member(T_j, T_k) ←
is_a_member($sp_i(s_1, s_2)$, S) *and*
is_a_member(c_j, s_2) *and*
is_a_member(c_k, s_2) *and*
$c_j \neq c_k$ *and*
teaching_time(c_j, T_j) *and*
teaching_time(c_k, T_k)

where $i = 1, \ldots, m$, $j = 1, \ldots, n$, $k = 1, \ldots, n$.

5) *Courses should be scheduled in the buildings desired for them.*

For all i, j,

is_a_member(r_i, R'') ←
is_a_member(c_j, C) *and*
teaching_room(c_j, R') *and*
prefer_building(c_j, b) *and*
building_rooms(b, R') *and*
is_a_member(r_i, R')

where $i = 1, \ldots, l$, $j = 1, \ldots, n$.

6) *Courses should not be scheduled on friday afternoons.*

For all j, k,

$t_k \leq 54$ *and* $t_k > 59$ ←
is_a_member(c_j, C) *and*
teaching_time(c_j, T_j) *and*
is_a_member(t_k, T_j)

where $j = 1, \ldots, m$, $k = 1, \ldots, 46$.

References

[1] Bloomfield, S.D., and McSharry, M.M., "Preferential course scheduling", *Interfaces* 9 (1979) 24–31.

[2] Brelaz, D., "New methods to color the vertices of a graph", *Communications of the ACM* 22/4 (1979) 251–256.

[3] Carter, M.W., "A Lagrangian relaxation approach to the classroom assignment problem", *INFOR* 27/2 (1989) 230–245.

[4] Cohen, J., "Constraint logic programming languages", *Communications of the ACM* 33/7 (1990) 52–68.

[5] DeWerra, D., "An introduction to timetabling", *European Journal of Operational Research* 19 (1985) 151–162.

[6] Dhar, V., and Ranganathan, N., "Integer programming vs. expert systems: An experimental comparison", *Communications of the ACM* 33/3 (1990) 323–336.

[7] Dincbas, M., Simonis, H.., and van Hentenryck, P., "Solving a cutting-stock program in constraint logic programming", *Proceedings of the Fifth International Conference and Symposium on Logic Programming*, Vol. 1, The MIT Press, Cambridge, MA, 1988.

[8] Even, S., Itai, A., and Shamir, A., "On the complexity of timetable and multicommodity flow problems", *SIAM Journal Computing* 5/4 (1976) 691–703.

[9] Feldman, R., and Golumbic, M.C., "Constraint satisfiability algorithms for interactive student scheduling", *Proceedings of Eleventh International Joint Conference on Artificial Intelligence* 1010–1015, Detroit, MI, 1989.

[10] Ferland, J.A., Roy, S., "Timetabling problem for university as assignment of activities to resources", *Computers and Operations Research* 12/2 (1985) 207–218.

[11] van Hentenryck, P., *Constraint Satisfaction in Logic Programming*, The MIT Press, Cambridge, MA, 1989.

[12] Karp, R.M., "Reducibility among combinatorial problems", *Complexity of Computer Computations*, Plenum Press, New York, 1972, 85–104.

[13] Lions, J., "The Ontario school scheduling program", *Computer Journal* 10 (1967) 14–21.

[14] Mulvey, J.M., "A classroom/time assignment Model", *European Journal of Operational Research* 9 (1982) 64–70.

[15] Peemoller, J., "A correction to Brelaz's modification of Brown's colouring algorithm" *Communications of the ACM* 26 (1983) 595–597.

[16] Robinson, J.A., "A machine-oriented logic based on the resolution principle", *Journal of the ACM* 12/1 (1965) 23–41.

[17] Schmidt, G., and Strohlein, T., "Timetable construction – an annotated bibliography", *Computer Journal* 23/4 (1980) 307–316.

[18] Selim, S.M., "An algorithm for producing course and lecture timetables", *Computers and Education* 7/2 (1983) 101–108.

[19] Tripathy, A., "A Lagrangian relaxation approach to course timetabling", *Journal of Research of National Bureau of Standards* 84 (1979) 489–506.

[20] White, G.M., and Wong, S.K.S., "Interactive timetabling in universities", *Computers and Education* 12/4 (1988) 521–529.

[21] White, G.M., and Haddad, M., "An heuristic method for optimizing examination schedules which have day and night courses", *Computers and Education* 7/4 (1983) 235–238.